SortBot

EECS3311

October 5, 2026

Shreeya Pai

221246616

1.1 Project Description

Problem and Motivation
SortBot is an AI agent that helps organize files for individuals with cluttered personal folders (Downloads, Documents, or Desktop). Consider a Downloads folder with 400 files: lecture slides, receipts, installers, screenshots. And several copies of the same PDF, many with names like “lec01 (3).pdf”. Finding anything takes time, and cleaning up by hand feels like a chore that keeps getting postponed. People often don’t know how to design a folder structure, don’t have time to build one, or often forget about old files that have been sitting there for years.

Target Users
Individuals with cluttered personal folders, such as students, professionals, and casual computer users, who lack the time or knowledge to organize them. No technical background is required, since the user can give instructions in plain language.

What The Agent Can Do
SortBot works only inside folders the user has explicitly authorized. It can:

- scan authorized folder and build an inventory of its files
- classify files using names, metadata, and short content snippets
- plan a folder structure and propose where each file should go
- let the user review and edit the plan, and only then execute it
- undo previous actions
- flag old, duplicate, or temporary files and move them to a To_Be_Deleted folder (never deletes anything)
- interpret natural language instructions, such as “put my tax documents in one folder”
- remember the user’s folder rules and preferences between sessions.

SortBot also shows agent behaviour: it plans a multi-step task, uses tools (file scanner, file mover, duplicate detector), keeps memory, and decides what to do from the user’s goal.

Why an AI agent and Not a Simple Script
A script can sort by extension or match keywords, but every rule has to be written in advance. It cannot decide where a file with an unclear name belongs, cannot turn instructions in natural language into concrete steps, and cannot adapt to how a particular person wants to organize things. An LLM can interpret unfamiliar content and instructions, an agent can break the job into steps and use tools to carry them out, and memory lets it improve over time.
Deterministic code is still used wherever safety matters: permissions, file operations, and undo.

Definition of Cleanup Candidates
A file is a cleanup candidate if it has not been modified in 12 months (user-configurable), is an exact duplicate of another file (matched by content hash), or is a temporary file (such as .tmp or installer leftovers). These are only flagged and moved, never deleted.

AI Model
SortBot will access the LLM through an LLMClient interface, so the model can be swapped without changing the rest of the system. The planned primary model is Google's Gemini Gemini 2.5 Flash-Lite (exact version to be confirmed in Stage 2), accessed through the free Google AI Studio API for its low cost and good quality at structured classification and planning. A local model run through Ollama is planned as an alternative for offline use and to keep file data on the user's machine. Both are implemented as interchangeable adapters, and the exact versions used will be recorded in Stage 2.

How the AI Interacts with the Rest of the System
The LLM never touches the file system. It receives a structured request from the planner and returns a proposed plan in a defined format (for example JSON). That output is parsed and checked by deterministic validators (is the path authorized, does the destination exist, will anything be overwritten) before it is shown to the user. Only after the user approves does a separate executor perform the moves, logging each one so it can be undone. In short: the LLM proposes, deterministic code verifies, the user approves, and the executor acts.

Interfaces and Architecture
SortBot provides a GUI (folder panel, plan review screen, chat panel, history view) and a CLI that offers the same major functions. Both are thin front ends over one shared core: an agent controller, planner, tool manager, memory manager, and LLM client. This keeps the logic in one place.

1.2 Feature Specification

F01: Authorize and Manage Allowed Folders

- **Description:** The user selects which folders SortBot may read and modify. All other paths are blocked.
- **User interaction:** In the GUI, an "Allowed Folders" panel with Add, Remove, and a list view. (The CLI has an equivalent command).
- **Input:** Folder path chosen by the user.
- **Output:** Updated allow-list, confirmation message.
- **AI involvement:** Deterministic.
- **Workflow:** User clicks Add, picks a folder, the system checks that it exists and is not a protected system location, then it stores the path.
- **Error/ alternative cases:** Path doesn't exist, path is a system directory, folder is already on the list, no read/write permission. In each case the system shows a specific message and does not update the list.

F02: Scan Folder and Build File Inventory

- **Description:** Traverses an authorized folder and records each file's name, type, size, dates, and path.
- **User interaction:** In the GUI, the user selects an authorized folder and clicks "Scan." A progress bar and a file table appear. The CLI has a scan command.
- **Input:** An authorized folder path with options such as including subfolders.
- **Output:** A file inventory (a list of file records) shown in the GUI.
- **AI involvement:** Deterministic.
- **Workflow:** The system verifies the folder is authorized, walks the directory, builds a

record for each file, skips anything unreadable, stores the inventory, and updates the progress display.
- **Error/ alternative cases:** Folder not authorized (rejected with a message); empty folder (shows "no files found"); unreadable or locked files (skipped and listed in a warnings area); very large folder (scan runs in the background with a cancel option); files changed during the scan (inventory notes the scan time).

F03: Content and Name Based Classification
- **Description:** Assigns each file category (for example Lecture Notes, Receipts, Photos) and a short reason, using name patterns, metadata, and text snippets.
- **User interaction:** After a scan, the user clicks "Classify." Each file in the table gets a category and a confidence label, and the user can correct any category manually.
- **Input:** The file inventory and for readable file types, a short extracted text snippet.
- **Output:** The inventory with a category, confidence, and reason for each file.
- **AI involvement:** Hybrid. Deterministic rules handle clear cases (known extensions, obvious name patterns), and the LLM handles ambiguous ones.
- **Workflow:** The classifier first applies deterministic rules. Files it can't settle are sent in batches to the LLM with their names and snippets. The response is parsed and validated, and low-confidence results are marked for user review.
- **Error/ alternative cases:** Unreadable or empty file (classified by name and type only); LLM returns malformed output (retry once, then fall back to "Unclassified"); LLM unavailable (rules only, with a warning); category not in the allowed list (rejected and marked unclassified); sensitive content (only snippet is ever sent).

F04: Generate Organization Plan
- **Description:** Proposes a folder structure and a destination for every classified file, taking saved preferences into account.
- **User interaction:** The user clicks "Generate Plan." A tree view shows proposed folders and the files that would go into each, but nothing is changed yet.
- **Input:** Classified inventory, saved folder rules and preferences, and optional user instructions
- **Output:** A plan – an ordered list of proposed actions (create folder, move file) with a destination for each file.
- **AI involvement:** AI-based, followed by deterministic validation.
- **Workflow:** The planner builds a prompt from the inventory and preferences, the LLM returns a structured plan, a parser converts it into action objects, and validators check that every path is inside an authorized folder, names don't collide, and no file is dropped or duplicated. The valid plan goes to the review screen.
- **Error/ alternative cases:** Invalid or incomplete LLM output (retry, then ask the user to adjust); plan references a path outside authorized folders (that action is removed and the user is told); name collision (auto-suffix or flag it); plan too large (split into batches); no files to organize (message shown).

F05: Review, Edit, and Approve Plan
- **Description:** Shows the proposed plan so the user can change it before anything happens. Nothing is executed without explicit approval.
- **User interaction:** In the GUI, a plan review screen with a tree of folders and files. The user can reject a single move, rename a target folder, drag a file to a different folder, or remove a folder. "Approve" and "Cancel" buttons sit at the bottom. The CLI shows the plan as a numbered list with accept, reject, and edit commands.
- **Input:** A validated plan and the user's edits.

- **Output:** An approved plan, or a cancelled one.
- **AI involvement:** Deterministic.
- **Workflow:** The plan is displayed, each edit is applied to the plan object, and the validators re-run after every edit. On Approve the plan is locked and passed to the executor.
- **Error/ alternative cases:** An edit creates a name collision or points outside an authorized folder (edit rejected with an explanation); the user rejects all moves (plan is cancelled); the files on disk changed since planning (user is warned and asked to rescan); the user closes the window mid-review (plan is discarded, not executed).

F06: Execute Plan

- **Description:** Carries out the approved plan: create folders and move files, recording each action as it goes.
- **User interaction:** After approval, a progress bar and a live log appear. The user can pause or cancel. The CLI prints progress lines.
- **Input:** An approved plan.
- **Output:** The reorganized files, a result summary (done, skipped, failed), and an action log used by undo and history.
- **AI involvement:** Deterministic.
- **Workflow:** For each action the executor rechecks authorization, performs the folder creation or move, appends an entry to the log, and notifies the display. At the end it shows a summary.
- **Error/ alternative cases:** File is locked or in use (skipped and listed); destination already exists (renamed with a suffix, never overwritten); permission denied; disk error mid-run (stop, keep the log of what was done, and offer undo); user cancels (completed moves stay logged and are undoable).

F07: Undo Previous Actions

- **Description:** Reverses earlier executed actions, either the last one or a whole run.
- **User interaction:** An “Undo” button and a history list in the GUI where the user picks a run or a single action. The CLI has undo and undo --run <id> commands.
- **Input:** The action log and the selected run or action.
- **Output:** Files restored to their original locations and the log updated.
- **AI involvement:** Deterministic.
- **Workflow:** The system loads the selected entries, reverses them in last-in first-out order, checks that each file is still where the log says, moves it back, and marks the entry as undone.
- **Error/ alternative cases:** File was moved or deleted by the user since; original location is now occupied (user chooses to rename or skip); folder created by SortBot is no longer empty (it is kept, not removed); nothing to undo (message shown); partial undo (the system reports which actions were not reversed).

F08: Detect Old and Duplicate Files and Move to To_Be_Deleted

- **Description:** Finds cleanup candidates (not modified for the configured period, exact duplicates, temporary files) and, after approval, moves them to a To_Be_Deleted folder. It never deletes anything.
- **User interaction:** A “Find Cleanup Candidates” button that opens a list with the reason for each file and checkboxes. The user confirms what to move. The CLI shows the same list with a confirm prompt.
- **Input:** The file inventory and settings (age threshold, patterns for temporary files).
- **Output:** A candidate list with reasons, and then the moved files in To_Be_Deleted.

- **AI involvement:** Hybrid. Age, hash-based duplicate detection, and temp patterns are deterministic. The LLM may add an opinion ("this looks like an outdated draft") that is shown as a suggestion only.
- **Workflow:** The detector computes ages and content hashes, groups duplicates and keeps one file per group as the original. The candidate list is shown. After the user confirms, the move goes through the same executor and logging as F06, so it is undoable.
- **Error/ alternative cases:** Unreadable file (cannot be hashed, so it is skipped and reported); a file marked as protected by a user rule (never flagged); no candidates found (message shown); a To_Be_Deleted folder already exists (reused); the LLM suggestion contradicts the rules (rules win and the suggestion is flagged for review).

F09: Natural Language Chat Instructions

- **Description:** The user types a request such as "put my tax documents in one folder" or "show me files I haven't touched in three years." The agent works out what is being asked, chooses which tools to use, and carries out the steps. Anything that would change files still goes through the plan and approval flow.
- **User interaction:** A chat panel in the GUI with a message box and a conversation view. The CLI has an equivalent chat command.
- **Input:** A natural language text message, the current inventory, and the saved preferences.
- **Output:** A reply in the chat, and one of: an answer, a clarifying question, or a new plan sent to the review screen.
- **AI involvement:** AI-based, with deterministic safeguards.
- **Workflow:** The controller sends the message to the planner. The LLM interprets the intent and returns a structured request (for example: filter files, generate a plan, run cleanup detection). A validator checks that it names a known tool and valid arguments, and the tool manager then runs the matching tool. The result is summarized back into the chat. The LLM never touches files directly.
- **Error/ alternative cases:** Ambiguous request (the agent asks one clarifying question instead of guessing); request refers to an unauthorized folder (refused and the user is told how to authorize it); the LLM names a tool that does not exist or gives invalid arguments (rejected, retried once, then the user is asked to rephrase); the user asks for deletion (explained that SortBot only moves files to To_Be_Deleted); LLM unavailable (chat shows an error and the non-chat features keep working); an instruction that conflicts with a saved rule (the agent points out the conflict and asks which takes priority).

F10: Remember User Preferences and Folder Rules

- **Description:** Stores rules and preferences the user has chosen, such as "receipts go in Finance/ Receipts," "never move files in Projects," or "flag files older than 18 months," and applies them in future plans and cleanups.
- **User interaction:** A "Preferences" screen in the GUI with a list of rules the user can add, edit, disable, or delete. The user can also say it in chat ("always put lecture slides under School"), and the agent proposes a rule for the user to confirm. The CLI has rules list/ add/ remove.
- **Input:** Rules created through the screen or confirmed from chat.
- **Output:** Saved rules and settings that persist between sessions, and rule information passed to the planner and cleanup detector.
- **AI involvement:** Hybrid. Storing, editing, and enforcing rules is deterministic. The LLM only helps turn a chat sentence into a proposed rule, and the user must confirm

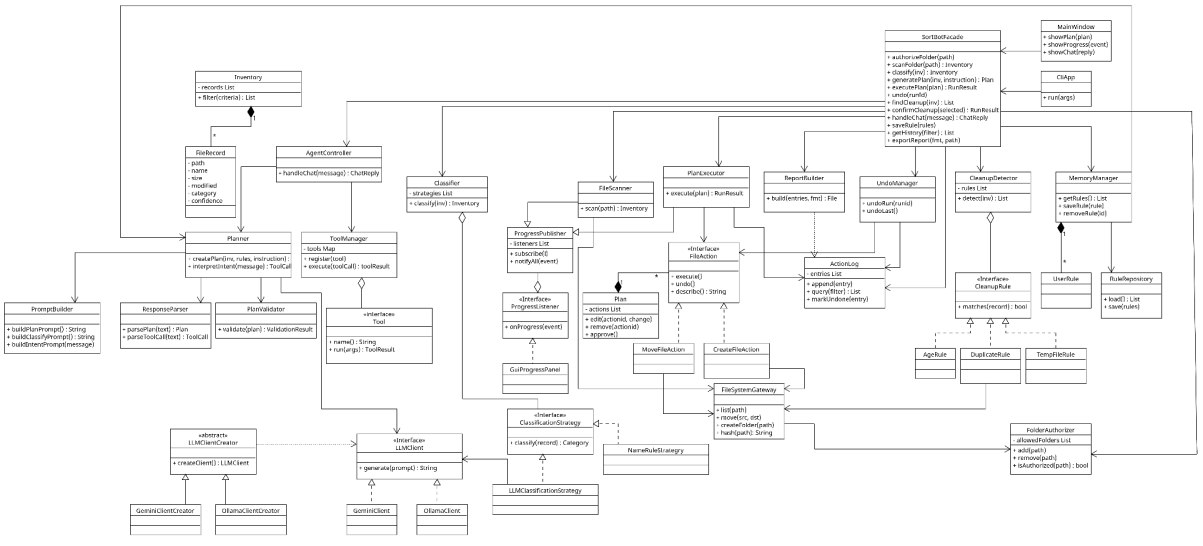
it.

- **Workflow:** A rule is saved to local storage. When a plan or cleanup scan runs, the memory manager retrieves the relevant rules and includes them in the planner's context. Validators then enforce protective rules (such as "never move") regardless of what the LLM proposes.
- **Error/ alternative cases:** Two rules conflict (the user is shown both and asked to resolve it); a rule points to a folder that is not authorized or no longer exists (flagged and disabled); the storage file is corrupted or missing (start with defaults and warn the user, keeping a backup copy); the LLM misreads a chat sentence as a rule (nothing is saved until the user confirms); a rule would be too broad (for example, matching every file) and is flagged before saving.

F11: Action History and Report Export

- **Description:** Keeps a permanent, searchable record of everything SortBot did (scans, plans, approvals, moves, undos, cleanup actions) and exports it as a report.
- **User interaction:** A "History" screen in the GUI with a list of runs that can be filtered by date, action type, or folder, and a detailed view per run. An "Export Report" button lets the user choose a format and save location. The CLI has history and export commands.
- **Input:** The action log, plus filter choices and export format (for example CSV or Markdown).
- **Output:** The on-screen history and an exported report file listing what changed, when, and the outcome of each action.
- **AI involvement:** Deterministic. An optional LLM-written plain language summary of a run could be added later, but is not required for the feature.
- **Workflow:** Each executed action is written to the log as it happens (by F06, F07, and F08). When the user opens History, the log is loaded and filtered. On export, a report builder formats the selected entries and writes the file.
- **Error/ alternative cases:** The log is empty (message shown); the log file is damaged (the readable portion is displayed with a warning); the export location is not writable or the disk is full (error shown and the user picks another location); a very large history (loaded in pages); an entry refers to a file that no longer exists (shown with a "missing" marker rather than failing).

2.1 Class Diagram



Full-size image: docs/diagrams/ClassDiagram.png.

Pattern: Command (undo and history)

- Problem: Moves and folder creations must be logged, replayed, and reversed.
- Participants: *FileAction* (interface: *execute()*, *undo()*, *describe()*), *CreateFolderAction*, *MoveFileAction* (concrete commands), *PlanExecutor* (invoker), *ActionLog* (history), *FileSystemGateway*(receiver).
- Why: Each action knows how to reverse itself, so undo is just "call undo() on logged actions in reverse order."
- Harder without it because undo would need big conditional on action type, and the log would be unstructured text instead of objects you can replay.

Pattern: Strategy (classification and cleanup rules)

- Problem: Files can be classified in several interchangeable ways (name rules, metadata, LLM), and cleanup uses several criteria (age duplicate, temp files).
- Participants: *ClassificationStrategy* (interface: *classify(FileRecord)*), *NameRuleStrategy*, *LLMClassificationStrategy*, *Classifier* (context); *CleanupRule* (interface: *matches(FileRecord)*), *AgeRule*, *DuplicateRule*, *TempFileRule*, *CleanupDetector* (context).
- Why: You can add or swap a method without editing the classifier. It also gives the clean "deterministic first, LLM for ambiguous files" flow.
- Harder without it because one giant classify method full of if/ else branches that is hard to test and extend.

Pattern: Adapter (LLM providers)

- Problem: Gemini and Ollama have different request formats, responses, and errors, but SortBot needs one interface.
- Participants: *LLMClient* (target interface: *generate(prompt)*), *GeminiClient* and *OllamaClient* (adapters), plus the vendor SDK or HTTP endpoint (adaptees)
- Why: The rest of the system never knows which model it is talking to, which is dependency inversion in practice.
- Harder without it because provider details would leak into the planner, and switching models would mean editing core code.

Pattern: Factory Method (creating the right client)

- Problem: Which *LLMClient* to build depends on configuration, and the planner shouldn't know how.
- Participants: *LLMClientCreator* (abstract: *createClient()*), *GeminiClientCreator*, *OllamaClientCreator*, products: *GeminiClient*, *OllamaClient*.
- Why: Adding a new provider means adding one creator class and no changes to existing logic.
- Harder without it because creation code with provider checks scattered around the app.

Pattern: Facade (one entry point for GUI and CLI)

- Problem: The GUI and CLI both need the same functions, and neither should know about a dozen internal classes.
- Participants: *SortBotFacade* (methods like *scanFolder()*, *generatePlan()*, *executePlan()*, *undo()*, *handleChat()*), subsystems behind it (*FolderAuthorizer*, *FileScanner*, *Classifier*, *AgentController*, *PlanExecutor*, *MemoryManager*, *ActionLog*).

- Why: It keeps the two interfaces thin and guarantees they behave identically.
- Harder without it because duplicated orchestration logic in the GUI and CLI drifts apart.

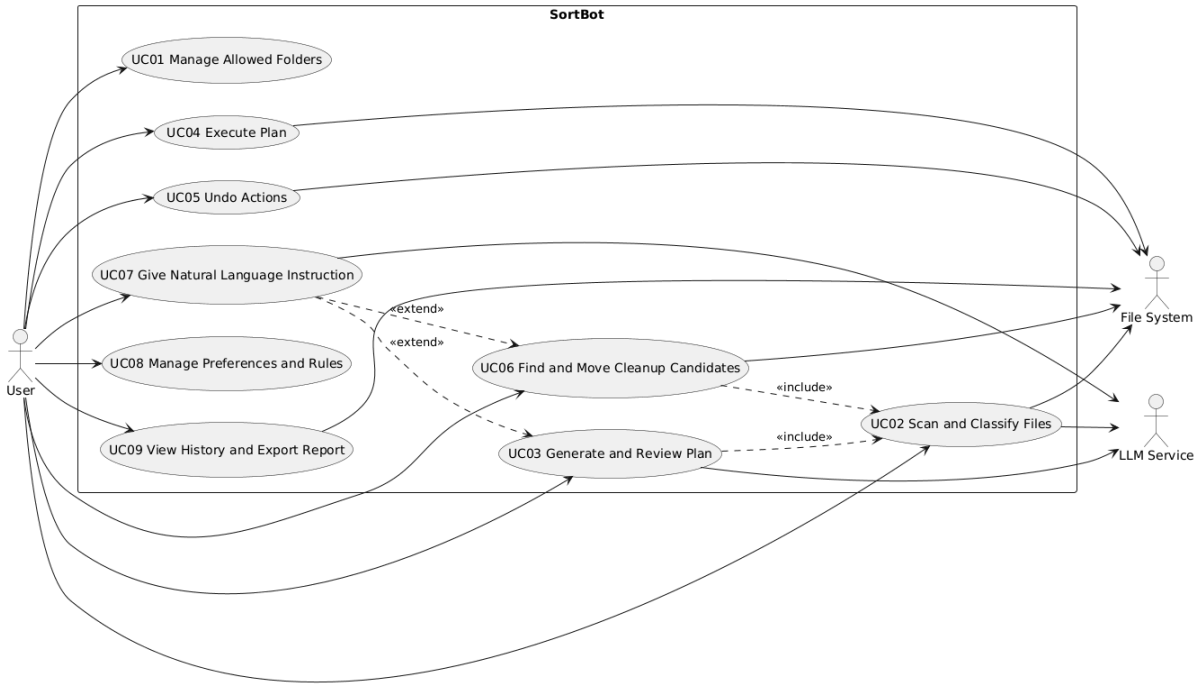
Pattern: Observer (progress update)

- Problem: Scanning and executing take time, and both the GUI and CLI need live progress without the core knowing which one is listening.
- Participants: *ProgressListener* (interface: *onProgress(event)*), *ProgressPublisher* (subject, extended by *FileScanner* and *PlanExecutor*), observers *GuiProgressPanel* and *CliProgressPrinter.*
- Why: The core reports progress without depending on any UI class.
- Harder without it because core classes would call GUI code directly, breaking separation of concerns and making testing difficult.

Layer	Class	Responsibility	Features
UI	MainWindow, CliApp	Collect input, show results	All
UI	GuiProgressPanel, CliProgressPrinter	Show progress (observes)	F02, F06
Facade	SortBotFacade	Single entry point	All
Agent	AgentController	Handles chat, routes requests	F09
Agent	Planner	Builds plans, interprets intent via LLM	F04, F09
Agent	ToolManager, Tool (interface)	Registers tools, validates and runs tool calls	F09
Agent	PromptBuilder, ResponseParser	Build prompts, parse and validate LLM output	F09
LLM	LLMClient, GeminiClient, OllamaClient, creators	Provider abstraction	F03, F04, F09
Domain	FileRecord, Inventory, Category	File data	F02, F03
Domain	Plan, PlanValidator	Ordered actions, rule checks, edits	F04, F05
Domain	FileAction,	Commands	F06, F07, F08

	CreateFolderAction, MoveFileAction		
Core	FolderAuthorizer	Allow-list, path checks	F01
Core	FileScanner	Build inventory	F02
Core	Classifier + strategies	Categorize files	F03
Core	PlanExecutor	Runs actions, log them	F06
Core	UndoManager	Reverse actions	F07
Core	CleanupDetector + rules	Find candidates	F08
Core	MemoryManager, UserRule, RuleRepository	Persist and apply rules	F10
Core	ActionLog, ReportBuilder	History and export	F11
Infra	FileSystemGateway	The only class that touches the disk	F02, F06, F07

2.2 Use-Case Diagram and Descriptions



UC01: Manage Allowed Folders

- **Actor(s):** User
- **Goal:** Control exactly which folders SortBot may read and change.
- **Preconditions:** SortBot is running.
- **Trigger:** User opens the Allowed Folders panel (or runs the CLI folder command).
- **Main Success Scenario:**
    1. User chooses Add and selects a folder.
    2. System calls authorizeFolder(path).
    3. FolderAuthorizer checks that the path exists, is readable and writable, and is not a protected system location.
    4. System adds the path to the allow-list and shows it in the list.
    5. (Optional) User removes a folder and the system updates the list.
- **Alternative/ exception flows:**
    - 3a. Path does not exist or is a system directory: system rejects it with a specific message.
    - 3b. No read or write permission: system rejects it and explains why.
    - 4a. Folder is already on the list: system shows a notice and makes no change.
- **Postconditions:** The allow-list reflects the user's choice and is saved. No other folder can be accessed.
- **Related features:** F01

UC02: Scan and Classify Files

- **Actor(s):** User; LLM Service; File System
- **Goal:** Build an inventory of an authorized folder and assign each file a category.

- **Preconditions:** At least one folder is authorized (UC01).
- **Trigger:** User selects a folder and clicks Scan, then Classify.
- **Main Success Scenario:**
    1. User selects an authorized folder and clicks Scan.
    2. System calls scanFolder(path); FileScanner lists files through FileSystemGateway and publishes progress events.
    3. System builds an Inventory of FileRecord objects and displays it.
    4. User clicks Classify and the system calls classify(inventory).
    5. Classifier applies NameRuleStrategy to the files it can settle.
    6. For ambiguous files, LLMClassificationStrategy sends names and short snippets to the LLM Service.
    7. System validates the responses and displays a category and confidence for every file.
- **Alternative/ exception flows:**
    - 2a. Folder not authorized: scan is refused.
    - 2b. Folder empty: system shows “no files found.”
    - 2c. Unreadable or locked file: skipped and listed in warnings.
    - 6a. LLM output is malformed: retry once, then mark the files “Unclassified.”
    - 6b. LLM unavailable: use rules only and warn the user.
- **Postconditions:** A classified Inventory exists in memory. No files have been changed.
- **Related features:** F02, F03

UC03: Generate and Review Plan

- **Actor(s):** User; LLM Service
- **Goal:** Get a proposed organization plan, edit it, and approve it.
- **Preconditions:** A classified inventory exists (includes UC02).
- **Trigger:** User clicks Generate Plan.
- **Main success scenario:**
    1. System calls generatePlan(inventory, instruction).
    2. Planner retrieves saved rules from MemoryManager and builds a prompt with PromptBuilder.
    3. Planner sends the prompt to LLMClient and ResponseParser converts the reply into a Plan of FileAction objects.
    4. PlanValidator checks authorization, collisions, and that no file is dropped or duplicated.
    5. System displays the plan as a folder tree.
    6. User edits the plan (rejects a move, renames a folder, reassigns a file); the system re-validates after each edit.
    7. User clicks Approve and the plan is locked for execution.
- **Alternative/exception flows:**
    - 3a. Invalid LLM output: retry once, then ask the user to adjust the request.
    - 4a. Action points outside authorized folders: that action is removed and the user is told.
    - 4b. Name collision: auto-suffix the name or flag it.
    - 6a. Edit breaks a rule: edit rejected with an explanation.
    - 7a. User cancels or closes: plan is discarded.
    - 7b. Files changed on disk since scanning: user is asked to rescan.
- **Postconditions:** An approved Plan is ready, or no plan exists. No files have been changed.
- **Related features:** F04, F05

UC04: Execute Plan

- **Actor(s):** User; File System
- **Goal:** Carry out an approved plan safely.
- **Preconditions:** An approved plan exists (UC03).
- **Trigger:** The plan is approved (or the user clicks Run).
- **Main success scenario:**
    1. System calls executePlan(plan).
    2. For each FileAction, PlanExecutor re-checks authorization and calls execute(), which uses FileSystemGateway.
    3. PlanExecutor appends an entry to ActionLog and publishes a progress event.
    4. GUI or CLI shows live progress.
    5. System shows a summary (done, skipped, failed).
- **Alternative/exception flows:**
    - 2a. File locked or in use: skipped and listed.
    - 2b. Destination exists: file is renamed with a suffix, never overwritten.
    - 2c. Permission denied or disk error: execution stops, completed actions stay logged, and undo is offered.
    - 4a. User cancels: remaining actions are not run; completed ones stay undoable.
- **Postconditions:** Files are reorganized and every action is logged.
- **Related features:** F06

UC05: Undo Actions

- **Actor(s):** User; File System
- **Goal:** Reverse the last action or a whole run.
- **Preconditions:** The action log contains at least one undoable entry.
- **Trigger:** User clicks Undo or selects a run in History.
- **Main success scenario:**
    1. User selects the last action or a run.
    2. System calls undo(runId).
    3. UndoManager loads the entries from ActionLog and calls undo() on each action in reverse order.
    4. Each file is moved back and the entry is marked undone.
    5. System shows a summary.
- **Alternative/exception flows:**
    - 3a. File no longer at the logged location: skipped and reported.
    - 3b. Original location is occupied: user chooses rename or skip.
    - 3c. A folder SortBot created is not empty: it is kept.
    - 2a. Nothing to undo: message shown.
- **Postconditions:** Reversible actions are undone and the log is updated.
- **Related features:** F07

UC06: Find and Move Cleanup Candidates

- **Actor(s):** User; File System
- **Goal:** Identify old, duplicate, or temporary files and move them to To_Be_Deleted.
- **Preconditions:** An inventory exists (includes UC02).
- **Trigger:** User clicks Find Cleanup Candidates.
- **Main success scenario:**
    1. System calls findCleanup(inventory).
    2. CleanupDetector applies AgeRule, DuplicateRule, and TempFileRule; protected files are excluded.
    3. System shows candidates with the reason for each.

4. User confirms or unchecks files.
5. System moves the confirmed files to To_Be_Deleted through the same execution and logging path as UC04.

- **Alternative/exception flows:**
  - 2a. A file can't be read for hashing: skipped and reported.
  - 3a. No candidates: message shown.
  - 5a. To_Be_Deleted already exists: it is reused.
  - 5b. Move fails for a file: reported, others continue.
- **Postconditions:** Confirmed files are in To_Be_Deleted and logged. Nothing is deleted.
- **Related features**: F08

UC07: Give Natural Language Instruction

- **Actor(s):** User; LLM Service
- **Goal:** Ask SortBot in plain language to do something.
- **Preconditions:** At least one authorized folder.
- **Trigger:** User sends a message in the chat panel (or chat in the CLI).
- **Main success scenario:**
  1. User types a request.
  2. System calls handleChat(message) and AgentController passes it to Planner.interpretIntent().
  3. The LLM returns a structured tool call.
  4. ResponseParser and ToolManager validate the tool name and arguments.
  5. ToolManager.execute() runs the tool (filter inventory, generate plan, find cleanup candidates).
  6. The result is summarized in the chat; any file-changing result goes to the plan review screen (UC03) or cleanup list (UC06).
- **Alternative/exception flows:**
  - 3a. Ambiguous request: the agent asks one clarifying question.
  - 4a. Unknown tool or invalid arguments: rejected, retried once, then the user is asked to rephrase.
  - 4b. Request refers to an unauthorized folder: refused, with instructions to authorize it.
  - 4c. User asks for deletion: explained that SortBot only moves files to To_Be_Deleted.
  - 3b. LLM unavailable: chat shows an error and other features continue.
- **Postconditions:** The user has an answer or a pending plan or list. Nothing changes without approval.
- **Related features:** F09

UC08: Manage Preferences and Rules

- **Actor(s):** User
- **Goal:** Save folder rules and preferences so future plans follow them.
- **Preconditions**: SortBot is running.
- **Trigger:** User opens Preferences, or asks in chat to remember a rule.
- **Main success scenario:**
  1. User adds a rule (for example, "receipts go to Finance/Receipts").
  2. System validates the rule (target folder authorized, not too broad).
  3. MemoryManager.saveRule() stores it through RuleRepository.
  4. The rule appears in the list and is used by Planner and CleanupDetector in later runs.

5. User can edit, disable, or delete rules.

- Alternative/exception flows:
  - 2a. Rule conflicts with an existing rule: user is shown both and asked to resolve.
  - 2b. Rule targets an unauthorized or missing folder: flagged and not saved.
  - 1a. Rule proposed from chat: nothing is saved until the user confirms.
  - 3a. Storage file is corrupted or missing: start with defaults, warn the user, and keep a backup.
- **Postconditions:** Rules persist between sessions.
- **Related features:** F10

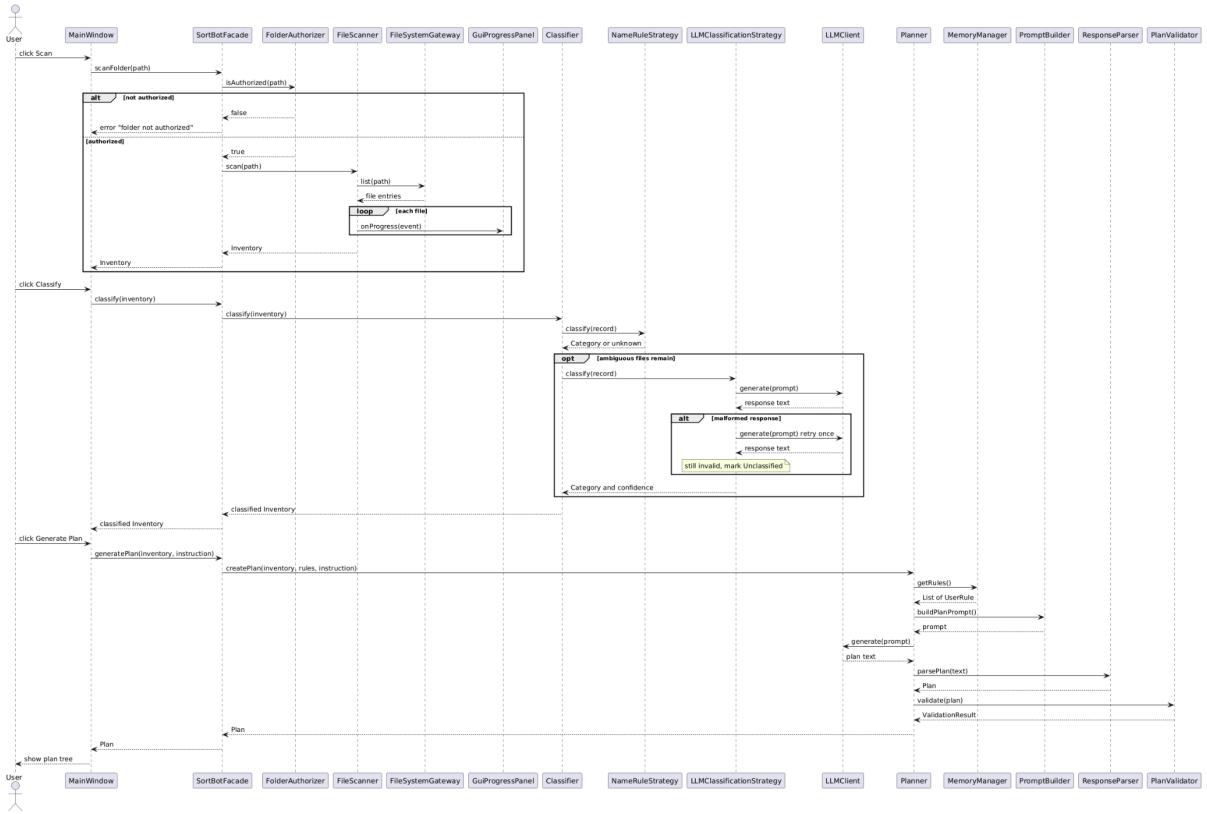
UC09: View History and Export Report

- **Actor(s):** User
- **Goal:** Review what SortBot has done and save a report.
- **Preconditions:** SortBot is running (history may be empty).
- **Trigger**: User opens History or clicks Export Report.
- **Main success scenario:**
  1. User opens History; system calls getHistory(filter).
  2. ActionLog returns entries, which the user filters by date, type, or folder.
  3. User opens a run to see its details.
  4. User clicks Export and chooses a format and location.
  5. System calls exportReport(fmt, path) and ReportBuilder writes the file.
- **Alternative/exception flows:**
  - 2a. History empty: message shown.
  - 2b. Log file damaged: readable entries shown with a warning.
  - 5a. Location not writable or disk full: error shown and the user picks another location.
  - 3a. Entry refers to a file that no longer exists: shown with a “missing” marker.
- **Postconditions**: Report file saved. History is unchanged.
- **Related features:** F11

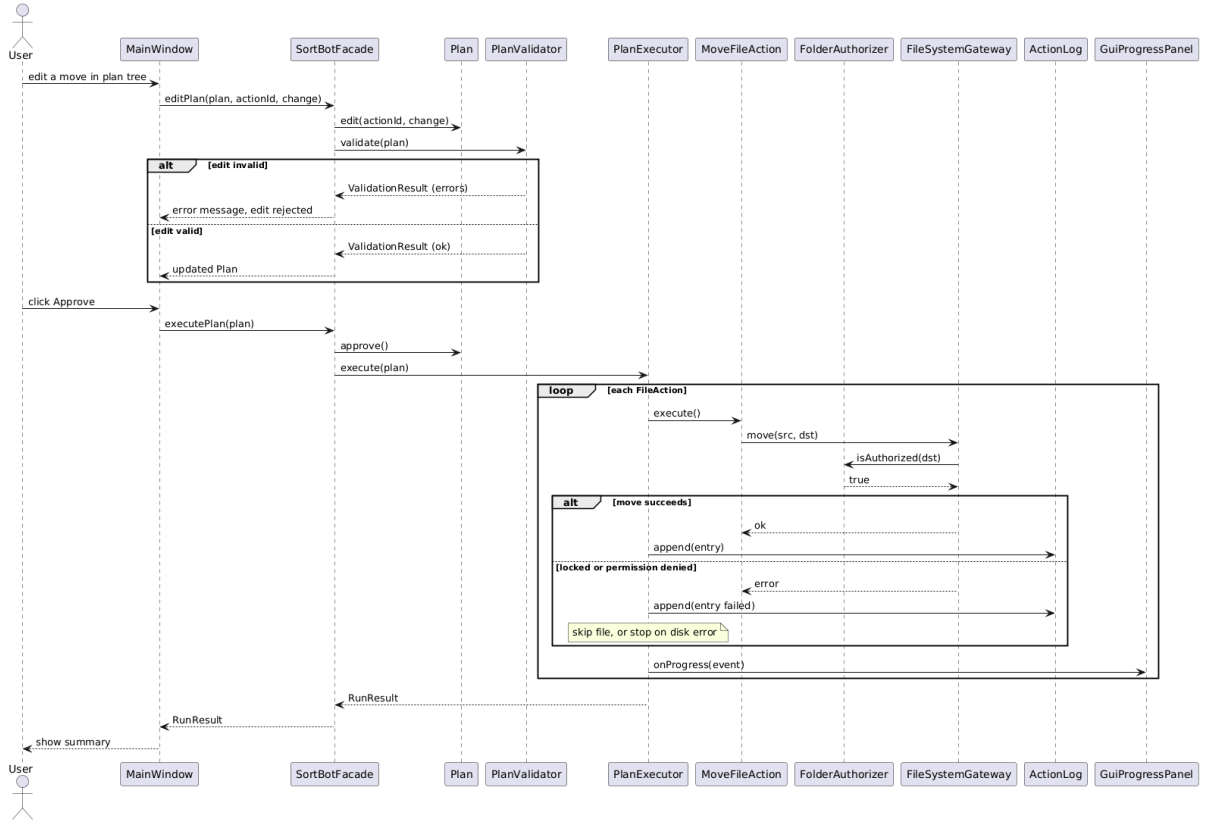
2.3 Sequence Diagram

Full-size images: docs/diagrams/ClassDiagram.png.

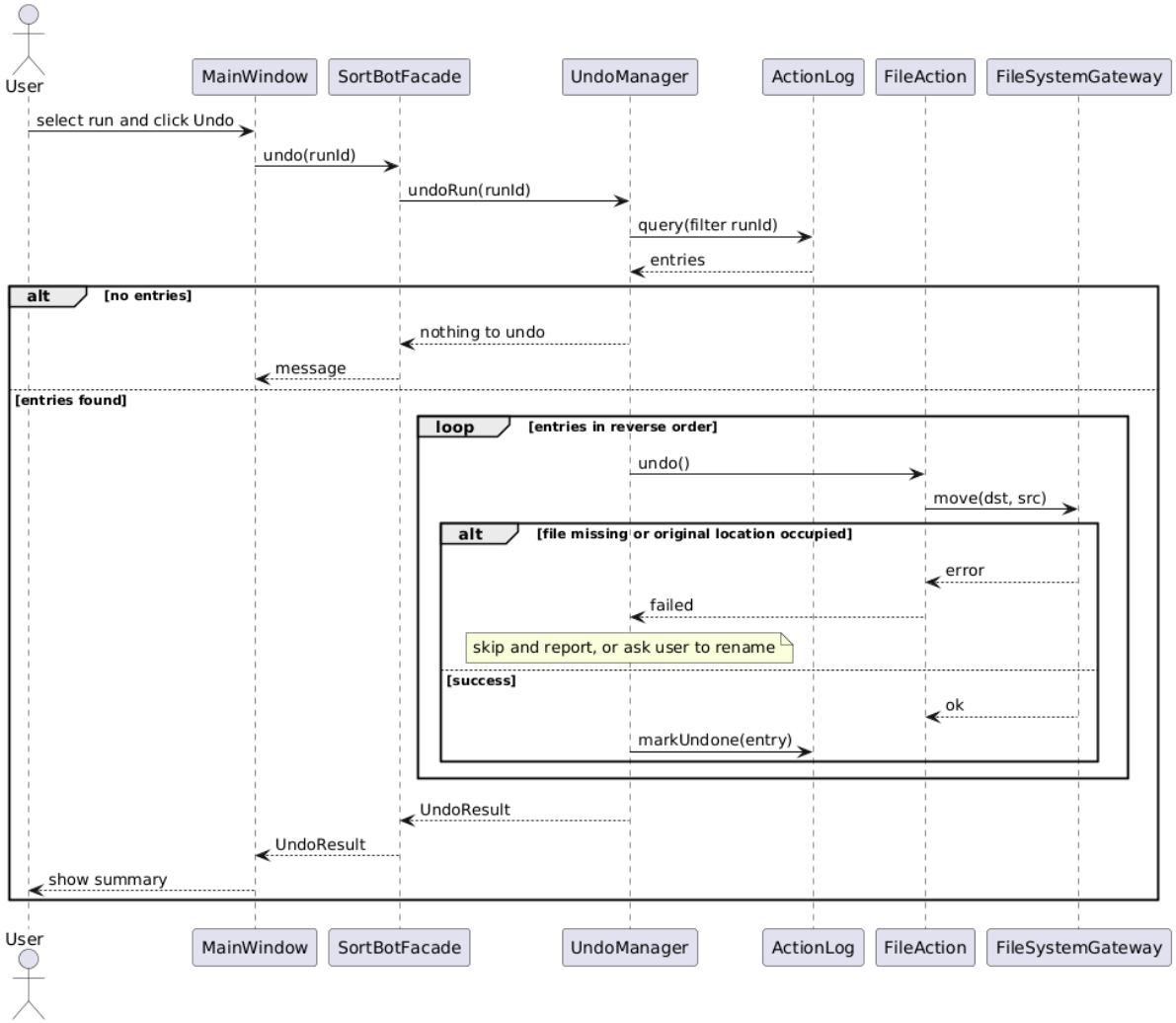
SD01 Scan, Classify, and Generate Plan (F02, F03, F04)



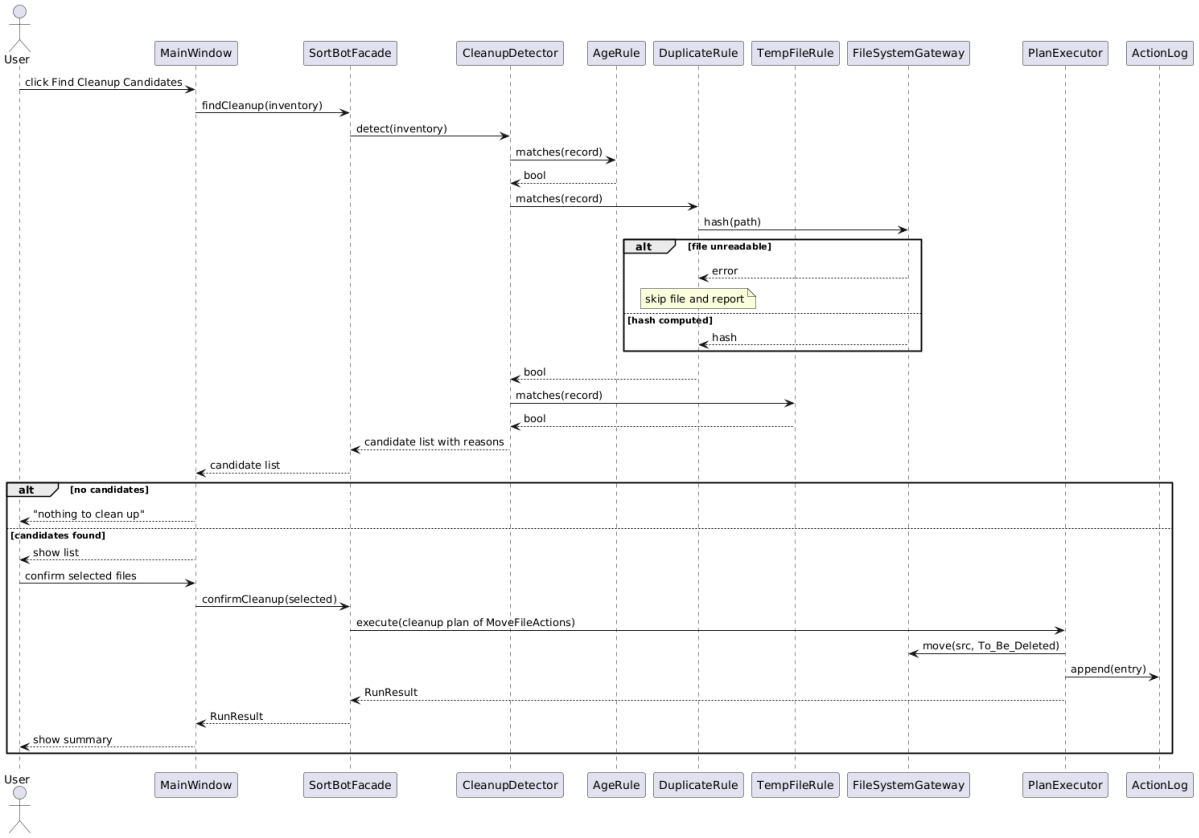
SD02 Review, Approve, and Execute Plan (F05, F06)



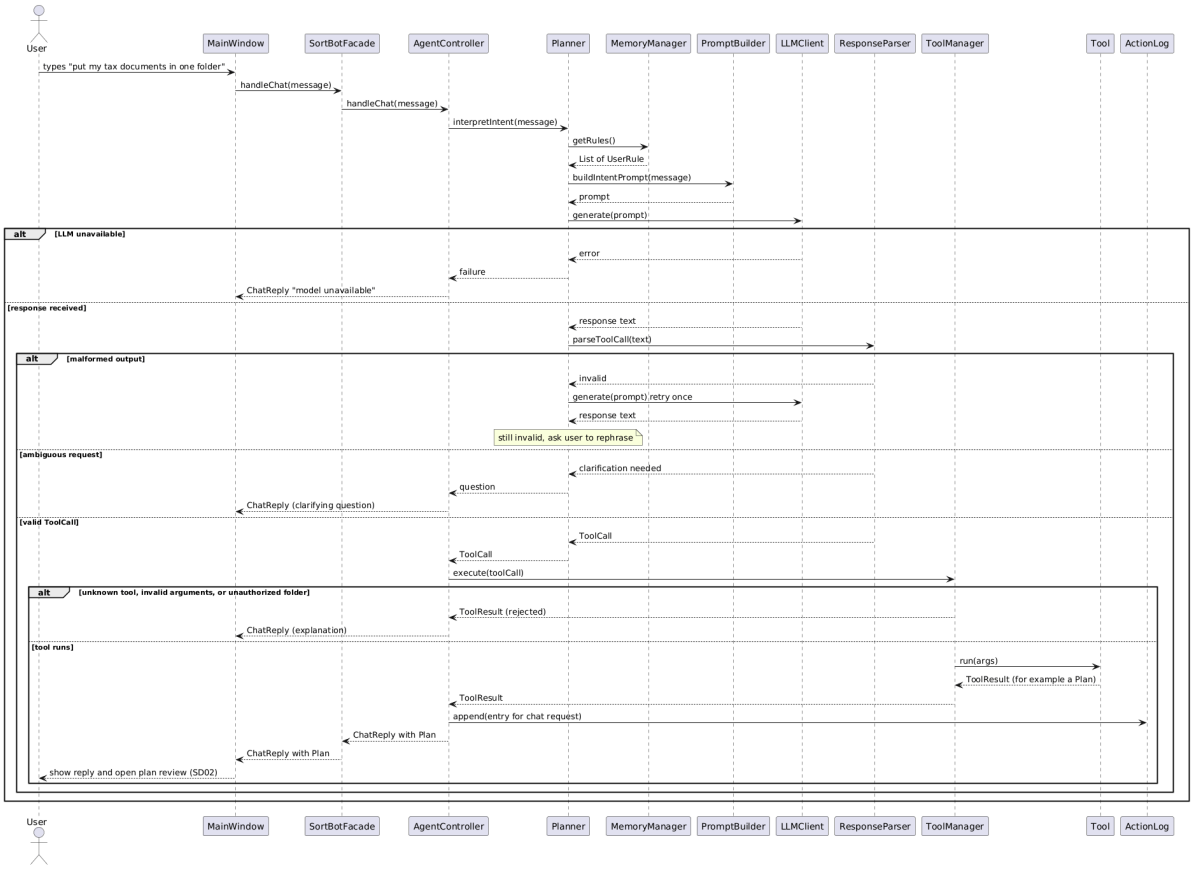
SD03 Undo Actions (F07)



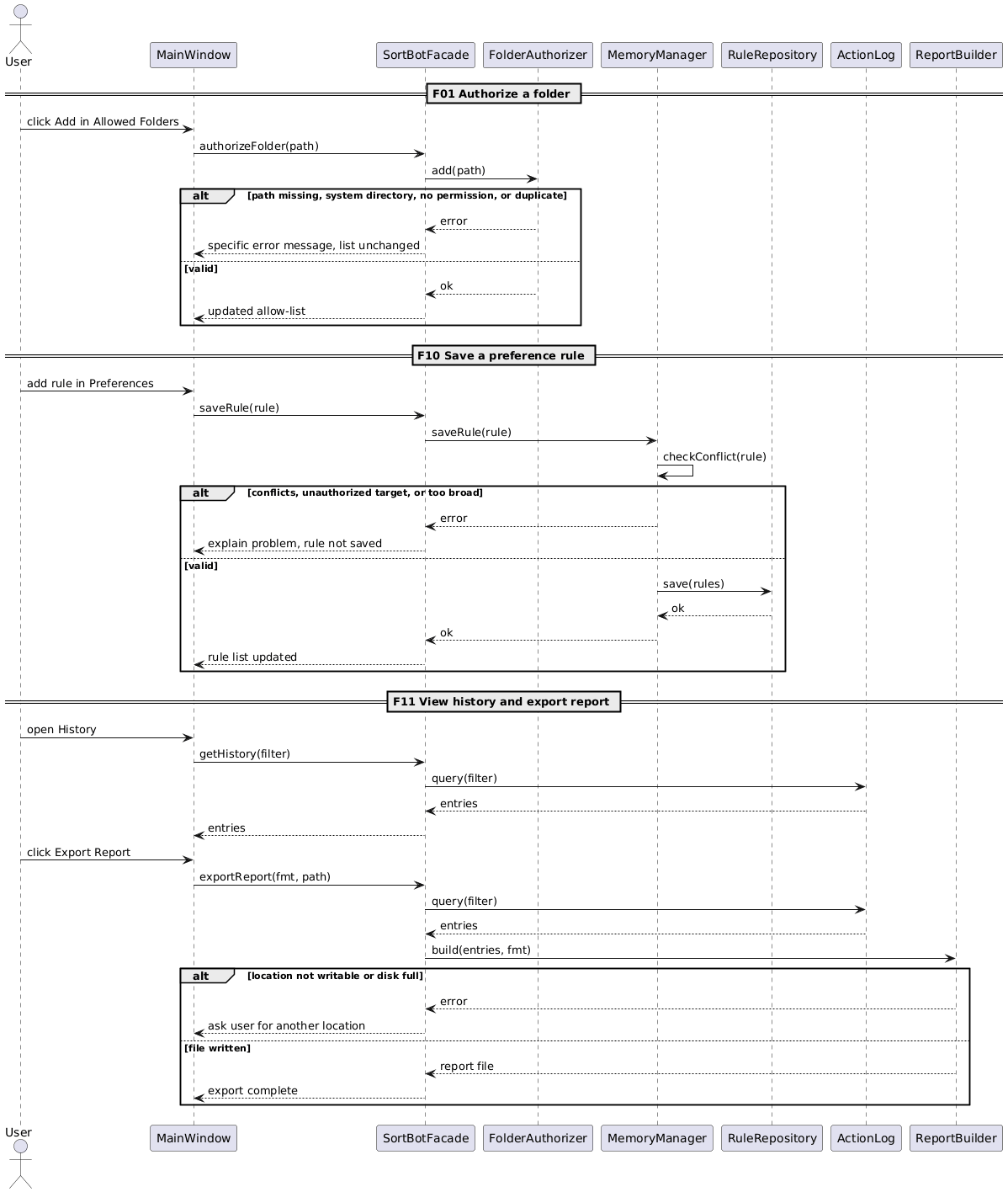
SD04 Find and Move Cleanup Candidates (F08)



SD05 Natural Language Instruction with Tool Call (F09)



SD06 Manage Folders, Save Rule, View History and Export Report (F01, F10, F11)



Task 3: Feature-to-Design Traceability

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Authorize and manage allowed folders	Deterministic	UC01 Manage Allowed Folders	MainWindow, CliApp, SortBotFacade, FolderAuthorizer, FileSystemGateway	authorizeFolder(), add(), remove(), isAuthorized()	SD01 (authorization check); add/remove follows the same facade-to-manager structure	Facade
F02	Scan a folder and build a file inventory	Deterministic	UC02 Scan and Classify Files	SortBotFacade, FileScanner, FileSystemGateway, Inventory, FileRecord, ProgressPublisher, GuiProgressPanel	scanFolder(), scan(), list(), onProgress()	SD01	Facade, Observer
F03	Content and name based classification	Hybrid	UC02 Scan and Classify Files	SortBotFacade, Classifier, ClassificationStrategy, NameRuleStrategy, LLMClassificationStrategy, LLMClient	classify(), generate()	SD01	Strategy, Adapter
F04	Generate an organization plan	AI-based (with deterministic validation)	UC03 Generate and Review Plan	SortBotFacade, Planner, PromptBuilder, ResponseParser, PlanValidator, MemoryManager, LLMClient, LLMClientCreator, Plan	generatePlan(), createPlan(), buildPlanPrompt(), parsePlan(), validate(), createClient()	SD01	Adapter, Factory Method, Facade
F05	Review, edit, and approve the plan	Deterministic	UC03 Generate and Review Plan	MainWindow, SortBotFacade, Plan, PlanValidator	editPlan(), edit(), remove(), approve(), validate()	SD02	Facade

F06	Execute the plan (create folders, move files)	Deterministic	UC04 Execute Plan	SortBotFacade, PlanExecutor, FileAction, MoveFileAction, CreateFolderAction, FileSystemGateway, FolderAuthorizer, ActionLog, ProgressPublisher	executePlan(), execute(), move(), createFolder(), append(), onProgress()	SD02	Command, Observer, Facade
F07	Undo previous actions	Deterministic	UC05 Undo Actions	SortBotFacade, UndoManager, ActionLog, FileAction, FileSystemGateway	undo(), undoRun(), query(), markUndone()	SD03	Command, Facade
F08	Detect old and duplicate files and move them to To_Be_Deleted	Hybrid (deterministic rules, optional LLM suggestion)	UC06 Find and Move Cleanup Candidates	SortBotFacade, CleanupDetector, CleanupRule, AgeRule, DuplicateRule, TempFileRule, PlanExecutor, FileSystemGateway	findCleanup(), detect(), matches(), hash(), confirmCleanup()	SD04	Strategy, Command
F09	Natural language chat instructions	AI-based (with deterministic safeguards)	UC07 Give Natural Language Instruction	MainWindow, SortBotFacade, AgentController, Planner, PromptBuilder, ResponseParser, ToolManager, Tool, LLMClient	handleChat(), interpretIntent(), buildIntentPrompt(), parseToolCall(), execute(), run()	SD05	Adapter, Facade
F10	Remember user preferences and folder rules	Hybrid	UC08 Manage Preferences and Rules	MemoryManager, RuleRepository, UserRule, Planner, CleanupDetector	getRules(), saveRule(), removeRule(), load(), save()	SD01 and SD05 (getRules used by the Planner); save/edit follows the same facade-to-manager structure	Repository, Facade

F11	Action history and report export	Deterministic	UC09 View History and Export Report	SortBotFacade, ActionLog, ReportBuilder	getHistory(), query(), append(), markUndone(), exportReport(), build()	SD02 and SD03 (log writes); export follows the same facade-to-manager structure	Facade

Task 4: How Each Feature Is Realized

F01: Authorize and Manage Allowed Folders

Related Use Case: UC01 Manage Allowed Folders

Related Sequence Diagram: SD01 (authorization check at the start of a scan)

Classes involved:

- MainWindow and CliApp: collect the folder path from the user.
- SortBotFacade: single entry point used by both interfaces.
- FolderAuthorizer: stores the allow-list and checks paths.
- FileSystemGateway: asks FolderAuthorizer before touching any path.

Important methods: SortBotFacade.authorizeFolder(), FolderAuthorizer.add(), FolderAuthorizer.remove(), FolderAuthorizer.isAuthorized()

Execution: When the user adds a folder, the interface calls authorizeFolder(path) on the facade. FolderAuthorizer checks that the path exists, is readable and writable, and is not a protected system location, then adds it to the allow-list. Every later file operation goes through FileSystemGateway, which calls isAuthorized() first, so unauthorized folders cannot be read or changed.

F02: Scan Folder and Build File Inventory

Related Use Case: UC02 Scan and Classify Files

Related Sequence Diagram: SD01

Classes involved:

- SortBotFacade: receives the scan request.
- FileScanner: walks the folder and builds the inventory.
- FileSystemGateway: lists files from disk.
- Inventory and FileRecord: hold the file data (an Inventory is composed of many FileRecords).
- ProgressPublisher and GuiProgressPanel: report progress (Observer).

Important methods: SortBotFacade.scanFolder(), FileScanner.scan(), FileSystemGateway.list(), ProgressListener.onProgress()

Execution: The facade checks the folder with FolderAuthorizer, then calls FileScanner.scan(). The scanner asks FileSystemGateway for the files, creates a FileRecord for each one, and publishes progress events that the GUI or CLI displays. It returns an Inventory. Unreadable files are skipped and reported.

F03: Content and Name Based Classification

Related Use Case: UC02 Scan and Classify Files

Related Sequence Diagram: SD01

Classes involved:

- Classifier: runs the classification strategies in order.
- NameRuleStrategy: classifies clear cases from names, extensions, and metadata.
- LLMClassificationStrategy: classifies ambiguous files using the language model.
- LLMClient (implemented by GeminiClient and OllamaClient): sends prompts to the model.

Important methods: Classifier.classify(), ClassificationStrategy.classify(), LLMClient.generate()

Execution: The Classifier first applies NameRuleStrategy. For files that remain ambiguous, LLMClassificationStrategy sends file names and short text snippets to the model through LLMClient. The response is validated against the allowed categories. If the output is malformed, the request is retried once, and if it still fails the file is marked Unclassified. Each FileRecord receives a category and a confidence value.

F04: Generate Organization Plan

Related Use Case: UC03 Generate and Review Plan

Related Sequence Diagram: SD01

Classes involved:

- Planner: coordinates plan creation.
- MemoryManager: supplies saved rules.
- PromptBuilder: builds the prompt from the inventory and rules.
- LLMClient and LLMClientCreator: call the model; the creator builds the configured client.
- ResponseParser: converts the model output into a Plan.
- PlanValidator: checks the plan.
- Plan: an ordered list of FileAction objects.

Important methods: SortBotFacade.generatePlan(), Planner.createPlan(), PromptBuilder.buildPlanPrompt(), LLMClient.generate(), ResponseParser.parsePlan(), PlanValidator.validate()

Execution: The facade calls createPlan(). The Planner gets the saved rules from MemoryManager, builds a prompt, and sends it to the LLM. ResponseParser turns the reply into a Plan of FileAction objects. PlanValidator checks that every path is authorized, names do not collide, and no file is dropped or duplicated. Invalid actions are removed or the model is asked once more. The validated Plan is returned for review. The model only proposes; it never touches files.

F05: Review, Edit, and Approve Plan

Related Use Case: UC03 Generate and Review Plan

Related Sequence Diagram: SD02

Classes involved:

- MainWindow: shows the plan as a folder tree and collects edits.
- SortBotFacade: passes edits and approval to the core.
- Plan: applies edits and records approval.
- PlanValidator: re-checks the plan after every edit.

Important methods: SortBotFacade.editPlan(), Plan.edit(), Plan.remove(), Plan.approve(), PlanValidator.validate()

Execution: When the user rejects a move, renames a target folder, or reassigns a file, the facade calls Plan.edit() and then PlanValidator.validate(). If the edit breaks a rule, it is rejected with an explanation. When the user clicks Approve, Plan.approve() locks the plan and it is handed to the executor. Nothing is executed without approval.

F06: Execute Plan

Related Use Case: UC04 Execute Plan

Related Sequence Diagram: SD02

Classes involved:

- PlanExecutor: runs the approved plan and acts as the invoker.
- FileAction, MoveFileAction, CreateFolderAction: command objects, each able to execute() and undo().
- FileSystemGateway: the only class that changes files, after asking FolderAuthorizer.
- ActionLog: records every action.
- ProgressPublisher and GuiProgressPanel: show live progress.

Important methods: SortBotFacade.executePlan(), PlanExecutor.execute(), FileAction.execute(), FileSystemGateway.move(), ActionLog.append()

Execution: PlanExecutor loops through the plan's actions and calls execute() on each. Each action uses FileSystemGateway, which re-checks authorization. After each action the executor appends an entry to ActionLog and publishes a progress event. Locked files are skipped, existing destinations are renamed with a suffix instead of overwritten, and on a disk error the run stops with completed actions still logged and undoable. The executor returns a

RunResult summary.

F07: Undo Previous Actions

Related Use Case: UC05 Undo Actions

Related Sequence Diagram: SD03

Classes involved:

- UndoManager: coordinates reversal.
- ActionLog: provides the logged actions.
- FileAction (MoveFileAction, CreateFolderAction): knows how to reverse itself through undo().
- FileSystemGateway: performs the reversal.

Important methods: SortBotFacade.undo(), UndoManager.undoRun(), ActionLog.query(), FileAction.undo(), ActionLog.markUndone()

Execution: The user selects a run or the last action. UndoManager loads the entries from ActionLog and calls undo() on each action in reverse order. Each file is moved back and the entry is marked undone. If a file has moved since, or the original location is occupied, that action is skipped and reported, and the user can choose to rename or skip. An UndoResult summary is returned.

F08: Detect Old and Duplicate Files and Move to To_Be_Deleted

Related Use Case: UC06 Find and Move Cleanup Candidates

Related Sequence Diagram: SD04

Classes involved:

- CleanupDetector: applies all cleanup rules to the inventory.
- CleanupRule (AgeRule, DuplicateRule, TempFileRule): interchangeable detection criteria.
- FileSystemGateway: computes content hashes for duplicate detection.
- PlanExecutor and MoveFileAction: move confirmed files to To_Be_Deleted.
- ActionLog: records the moves so they can be undone.

Important methods: SortBotFacade.findCleanup(), CleanupDetector.detect(), CleanupRule.matches(), FileSystemGateway.hash(), SortBotFacade.confirmCleanup()

Execution: The facade calls detect(), which applies each rule to every file. AgeRule checks the last modified date, DuplicateRule compares content hashes, and TempFileRule checks name patterns. Files protected by user rules are excluded. The user sees the candidates with reasons and confirms which to move. The confirmed files are moved to To_Be_Deleted through the same execution and logging path as F06, so the moves are undoable. Nothing is ever deleted.

F09: Natural Language Chat Instructions

Related Use Case: UC07 Give Natural Language Instruction

Related Sequence Diagram: SD05

Classes involved:

- MainWindow (chat panel) and SortBotFacade: receive the message.
- AgentController: coordinates the agent steps.
- Planner: interprets the intent using the LLM.
- PromptBuilder and ResponseParser: build the prompt and parse the model's structured reply.
- ToolManager and Tool: validate and run the chosen tool.
- LLMClient: communicates with the model.

Important methods: SortBotFacade.handleChat(), AgentController.handleChat(), Planner.interpretIntent(), ResponseParser.parseToolCall(), ToolManager.execute(), Tool.run()

Execution: The facade passes the message to AgentController, which asks the Planner to interpret it. The Planner builds a prompt (including saved rules), the LLM returns a structured tool call, and ResponseParser validates it. If the request is ambiguous, the agent asks a clarifying question. If the tool name or arguments are invalid, or the request refers to an unauthorized folder, ToolManager rejects it. Otherwise ToolManager runs the tool (for example, generate a plan or find cleanup candidates), and any result that would change files is sent to the plan review screen for approval.

F10: Remember User Preferences and Folder Rules

Related Use Case: UC08 Manage Preferences and Rules

Related Sequence Diagram: SD01 and SD05 (rules retrieved by the Planner)

Classes involved:

- MemoryManager: stores, retrieves, and removes rules.
- RuleRepository: saves rules to local storage and loads them.
- UserRule: one saved rule.
- Planner and CleanupDetector: use the rules in later runs.

Important methods: MemoryManager.saveRule(), MemoryManager.getRules(), MemoryManager.removeRule(), RuleRepository.load(), RuleRepository.save()

Execution: When the user adds a rule through the Preferences screen (or confirms one proposed from chat), the facade calls saveRule(). The rule is validated (target folder authorized, not too broad, no conflict) and saved through RuleRepository so it persists between sessions. In later runs, Planner.createPlan() calls getRules() and includes the rules in its prompt, and protective rules such as "never move" are enforced by the validators regardless of what the model proposes. If the storage file is corrupted, defaults are loaded and the user is warned.

F11: Action History and Report Export

Related Use Case: UC09 View History and Export Report

Related Sequence Diagram: SD02 and SD03 (entries written to the log)

Classes involved:

- ActionLog: stores every action with its outcome.
- ReportBuilder: formats entries into an exported report.
- SortBotFacade and MainWindow (History screen): query and display the log.

Important methods: SortBotFacade.getHistory(), ActionLog.query(), ActionLog.append(), SortBotFacade.exportReport(), ReportBuilder.build()

Execution: PlanExecutor and UndoManager write entries to ActionLog as actions happen. When the user opens History, getHistory(filter) queries the log by date, action type, or folder. On export, the facade calls ReportBuilder.build() with the selected entries and format, which writes the file to the location the user chose. An empty log shows a message, a damaged log shows the readable entries with a warning, and an unwritable export location prompts the user to pick another.